
Departamento de Energía y Automatización
Robótica Industrial

PRÁCTICA Nº 3

Actuadores

Asignatura: Robótica Industrial (FPTEN27)

Profesores: José Barriola · Leonardo Delgado · José Montilva

Laboratorio de Robótica Industrial
Caracas, 10 de marzo de 2026

Guía de Práctica de Laboratorio

Práctica #3 — Actuadores

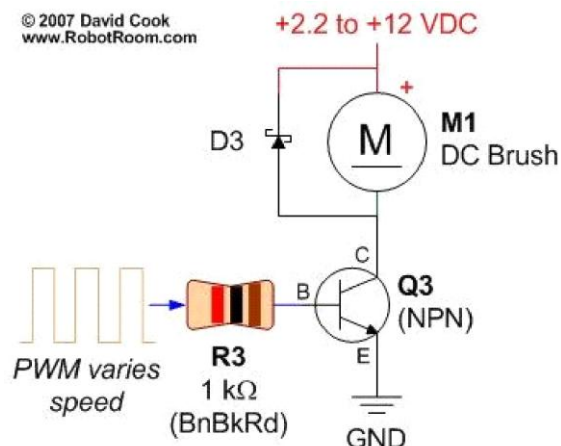
Objetivo General

Comprender el principio de operación y el método de control de los tres tipos de actuadores eléctricos fundamentales en robótica: motores DC, servomotores y motores paso a paso. Se trabajará con señales PWM, comunicación serial y secuencias de conmutación de bobinas.

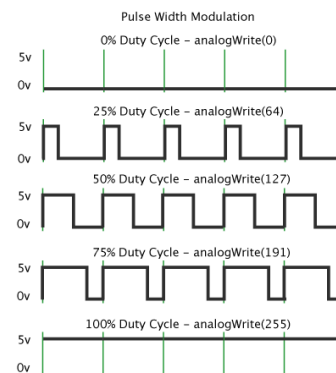
Ejercicio #1 Motor de Corriente Continua (DC) con Transistor

Un motor DC convierte energía eléctrica en movimiento rotacional. Su velocidad es proporcional al voltaje aplicado. Sin embargo, el Arduino no puede alimentar un motor directamente desde sus pines de salida: la corriente máxima de un pin digital es ~40 mA, mientras que un motor pequeño puede requerir 200–500 mA. Por eso se usa un transistor como interruptor de potencia.

© 2007 David Cook
www.RobotRoom.com



Esquemático: transistor NPN (Q3) como driver, diodo de protección (D3)



Ciclos de trabajo PWM — `analogWrite(0)` a `analogWrite(255)`

¿Por qué el transistor y el diodo?

El transistor NPN (Q3) actúa como un interruptor controlado por la señal PWM del Arduino. Cuando el pin recibe una señal, el transistor conduce y deja pasar la corriente de la fuente externa al motor. El diodo D3 (diodo de rueda libre o flyback) protege el transistor: cuando el motor se apaga, sus bobinas generan un pulso de voltaje inverso que podría dañar el transistor — el diodo lo cortocircuita de forma segura.

⚠ Concepto clave — No conectar el motor directamente al Arduino

- El pin 3 del Arduino UNO solo puede suministrar ~40 mA. Un motor DC pequeño puede consumir 5-10 veces más.
- Sin el transistor, el motor no giraría correctamente y podrías dañar el Arduino.
- El pin 3 sí es PWM — puedes verificarlo en la serigrafía del Arduino buscando el símbolo ~ junto al número del pin.
- Asegúrate de comprender la función del diodo D3 y la razón por la que el motor no puede conectarse directamente al pin del Arduino — son conceptos fundamentales del circuito.

Programa

El programa lee un carácter (0–9) por el Monitor Serial y mapea ese valor al rango 0–255 para usar con `analogWrite()`:

```
const int motorPin = 3;    // Pin PWM del motor (verificar símbolo ~ en el
                             Arduino)

void setup() {
  Serial.begin(9600);
  Serial.println("Control de Velocidad de Motor DC!");
  Serial.println("Introduzca velocidad del 0 al 9");
}

void loop() {
  if (Serial.available()) {
    char ch = Serial.read();
    if (isDigit(ch)) {
      int speed = map(ch, '0', '9', 0, 255); // '0'→0, '9'→255
      analogWrite(motorPin, speed);
      Serial.print("Duty cycle = ");
      Serial.print(speed);
      Serial.println(" / 255 (no son RPM: es la proporción del ciclo de
trabajo PWM que es de 8 bits)");
    } else {
      Serial.print("Carácter no reconocido: ");
      Serial.println(ch);
    }
  }
}
```

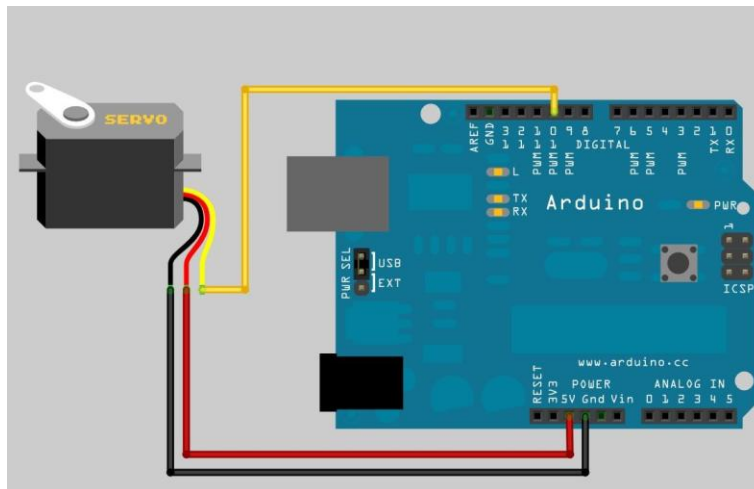
⚠ ¿Qué observar durante el ejercicio?

- Envía '0' por el Monitor Serial → el motor debe detenerse. Envía '9' → velocidad máxima.
- Reflexiona: cuando envías '5', ¿el motor recibe realmente 2.5 V constantes? Observa qué tipo de señal genera `analogWrite()` y cómo eso se traduce en velocidad.

Ejercicio #2 Servomotor — Control de Posición Angular

Un servomotor es un actuador que puede posicionarse en un ángulo específico y mantenerlo. A diferencia del motor DC, no controla velocidad continua sino posición angular (típicamente 0° a 180°). Internamente contiene un motor DC, una caja reductora de engranajes, un potenciómetro

para medir la posición actual, y un circuito de control — todo en un solo módulo. Es, por tanto, un sistema de lazo cerrado.



Conexión del servomotor: señal → pin 9, VCC → 5V, GND → GND (cables: naranja/blanco=señal, rojo=VCC, negro/marrón=GND)

△ Lazo cerrado — ¿dónde ocurre exactamente?

- El Arduino solo envía la posición deseada (setpoint) por el pin de señal.
- El circuito interno del servo compara esa posición con la posición real (medida por su potenciómetro interno).
- Si hay error, el motor interno corrige hasta que las dos posiciones coincidan.
- Por eso no necesitas retroalimentar nada desde el servo al Arduino: el lazo cerrado es interno.

Problema #1 — Control en barrido automático (Sketch: Sweep)

Este sketch mueve el servo de 0° a 180° y regresa, continuamente:

```
#include <Servo.h>

Servo myservo;    // Objeto servo - gestiona la señal de control
int pos = 0;

void setup() {
  myservo.attach(9);  // Asocia el objeto al pin 9
}

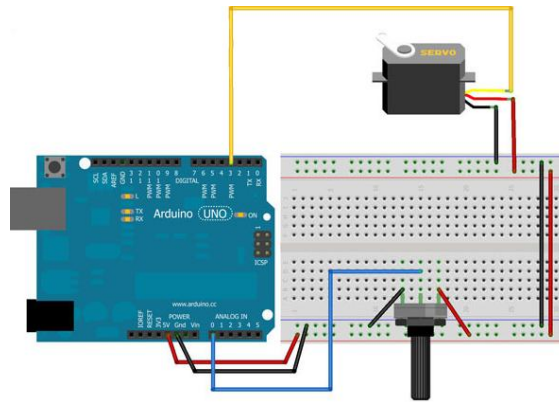
void loop() {
  for (pos = 0; pos <= 180; pos++) {    // 0° → 180°
    myservo.write(pos);
    delay(15);                          // 15 ms para que el servo llegue
  }
  for (pos = 180; pos >= 0; pos--) {    // 180° → 0°
    myservo.write(pos);
    delay(15);
  }
}
```

}

⚠ ¿Por qué no usar analogWrite() directamente?

- analogWrite() genera PWM a ~490 Hz con valores 0–255.
- Un servo necesita una señal de control muy específica: pulsos entre 1 ms y 2 ms, repetidos cada 20 ms (50 Hz).
- La librería Servo.h genera exactamente esa señal. myservo.attach(9) configura el pin 9 para ese propósito.
- Si intentaras usar analogWrite(9, 90) para poner el servo en 90°, el servo no lo entendería — la frecuencia y los tiempos son diferentes.

Problema #2 — Control por potenciómetro (Sketch: Knob)



Potenciómetro en A0 controla la posición del servo en pin 9

Este sketch lee un potenciómetro y convierte el valor analógico (0–1023) en un ángulo (0°–180°):

```
#include <Servo.h>

Servo myservo;
int potpin = 0;    // Potenciómetro en A0
int val;

void setup() {
  myservo.attach(9);
}

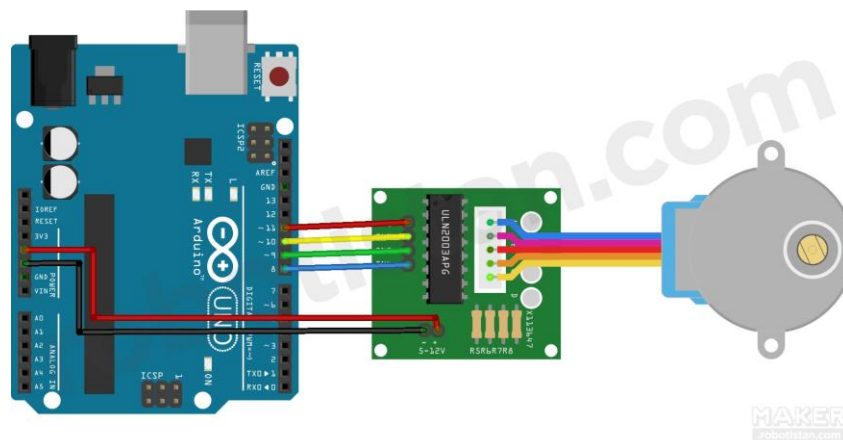
void loop() {
  val = analogRead(potpin);    // Lee 0-1023
  val = map(val, 0, 1023, 0, 180); // Convierte a 0°-180°
  myservo.write(val);          // Envía posición
  delay(15);
}
```

⚠ ¿Qué pasa si escribes myservo.write(200)?

- El rango válido del servo es 0° – 180° . Si envías 200, el servo intentará ir más allá de su límite mecánico.
- Esto puede hacer que el motor interno forcejee contra los topes, genere calor y desgaste excesivo.
- En el peor caso, puede dañar los engranajes o el potenciómetro interno del servo.
- Ten presente que enviar valores fuera del rango válido tiene consecuencias físicas reales sobre el hardware — no es solo un error de software.

Práctica #3 — Motor Paso a Paso Unipolar

Un motor paso a paso (stepper) se mueve en incrementos discretos — cada 'paso' corresponde a un ángulo fijo de giro. A diferencia del servo, no tiene sensor de posición interno: el control es en lazo abierto. La posición se estima contando los pasos dados desde una referencia conocida. Se usan en impresoras 3D, plotters y brazos robóticos donde se necesita precisión repetible sin sensor de retroalimentación.



Conexión: Motor unipolar → Driver ULN2003A → pines 8, 9, 10, 11 del Arduino

¿Qué es el chip ULN2003A?

El ULN2003A es un arreglo de transistores Darlington. Cumple la misma función que el transistor del motor DC: permite que el Arduino controle cargas de mayor corriente. Cada uno de sus 4 canales maneja una bobina del motor. Sin él, el Arduino no podría suministrar la corriente necesaria para las bobinas del stepper.

▲ Diferencia clave entre motor DC, servo y stepper

- Motor DC: control de velocidad continua, lazo abierto, no conoce su posición.
- Servomotor: control de posición angular (0° – 180°), lazo cerrado interno, rango limitado.
- Motor paso a paso: control de posición por conteo de pasos, lazo abierto, rango ilimitado de giro.
- Esta comparación entre los tres actuadores es fundamental para entender cuándo usar cada uno en una aplicación robótica real.

Modos de control — Resumen

Existen tres modos de conmutación para el motor unipolar. La siguiente tabla resume las bobinas activas en cada paso de cada modo:

Modo	Paso	IN1	IN2	IN3	IN4
Wave Drive	1	●	○	○	○
	2	○	●	○	○
	3	○	○	●	○
	4	○	○	○	●
Full Step	1	●	●	○	○
	2	○	●	●	○
	3	○	○	●	●
	4	●	○	○	●
Half Step	1	●	○	○	○
	2	●	●	○	○
	3	○	●	○	○
	4	○	●	●	○
	5	○	○	●	○
	6	○	○	●	●
	7	○	○	○	●
	8	●	○	○	●

● bobina activa ○ bobina inactiva

Ejercicio #3.1 Secuencia Wave Drive

Con este modo se energiza una sola bobina a la vez. Es el modo más simple pero el que menos torque produce, ya que en cada instante solo un campo magnético actúa sobre el rotor.

Versión A — Con matriz (Unipolar_Step_Wave)

La secuencia de pasos se almacena en una matriz bidimensional. El bucle for recorre las filas de la matriz y aplica cada fila a los pines de salida:

```

#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

// Cada fila = un paso; cada columna = una bobina (IN1 a IN4)
int paso[4][4] = {
    {1, 0, 0, 0}, // Paso 1: solo IN1
    {0, 1, 0, 0}, // Paso 2: solo IN2
    {0, 0, 1, 0}, // Paso 3: solo IN3
    {0, 0, 0, 1}  // Paso 4: solo IN4
};

```

```
};

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

void loop() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(IN1, paso[i][0]);
    digitalWrite(IN2, paso[i][1]);
    digitalWrite(IN3, paso[i][2]);
    digitalWrite(IN4, paso[i][3]);
    delay(10);    // Controla la velocidad - delay menor = giro más rápido
  }
}
```

Versión B — Hardcodeada (Unipolar_Step_Wave_HC)

Esta versión escribe los estados de cada bobina de forma explícita en el código, sin usar matriz. Cada paso es una secuencia de cuatro llamadas a digitalWrite(). Es más larga pero permite ver directamente qué bobina enciende en cada momento:

```
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

void loop() {
  // Paso 1: solo IN1
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  delay(10);

  // Paso 2: solo IN2
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  delay(10);

  // Paso 3: solo IN3
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  delay(10);

  // Paso 4: solo IN4
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  delay(10);
}
```

Ejercicio #3.2 Secuencia Full Step

Con este modo se energizan dos bobinas adyacentes simultáneamente. Los dos campos magnéticos se suman, lo que produce mayor torque que el Wave Drive. Es el modo más usado en aplicaciones prácticas con motores unipolares.

Versión A — Con matriz (Unipolar_Step_Full_Mode)

```
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

// Cada fila = un paso; dos bobinas activas por paso
int paso[4][4] = {
  {1, 1, 0, 0}, // Paso 1: IN1 + IN2
  {0, 1, 1, 0}, // Paso 2: IN2 + IN3
  {0, 0, 1, 1}, // Paso 3: IN3 + IN4
  {1, 0, 0, 1} // Paso 4: IN4 + IN1
};

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

void loop() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(IN1, paso[i][0]);
    digitalWrite(IN2, paso[i][1]);
    digitalWrite(IN3, paso[i][2]);
    digitalWrite(IN4, paso[i][3]);
    delay(10);
  }
}
```

Versión B — Hardcodeada (Unipolar_Step_Full_Mode_HC)

```
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

void loop() {
  // Paso 1: IN1 + IN2
  digitalWrite(IN1, HIGH); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  delay(10);

  // Paso 2: IN2 + IN3
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  delay(10);
}
```

```
// Paso 3: IN3 + IN4
digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
digitalWrite(IN3, HIGH); digitalWrite(IN4, HIGH);
delay(10);

// Paso 4: IN4 + IN1
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
delay(10);
}
```

Ejercicio #3.3 Secuencia Half Step

El modo Half Step combina los dos modos anteriores alternando pasos de una sola bobina con pasos de dos bobinas. El resultado es que el motor da el doble de pasos por vuelta (mayor resolución angular) y el movimiento es más suave. El torque es intermedio entre Wave y Full Step.

Versión A — Con matriz (Unipolar_Step_Half)

```
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

// 8 pasos por ciclo - alterna 1 bobina / 2 bobinas
int paso[8][4] = {
  {1, 0, 0, 0}, // Paso 1: solo IN1
  {1, 1, 0, 0}, // Paso 2: IN1 + IN2
  {0, 1, 0, 0}, // Paso 3: solo IN2
  {0, 1, 1, 0}, // Paso 4: IN2 + IN3
  {0, 0, 1, 0}, // Paso 5: solo IN3
  {0, 0, 1, 1}, // Paso 6: IN3 + IN4
  {0, 0, 0, 1}, // Paso 7: solo IN4
  {1, 0, 0, 1} // Paso 8: IN4 + IN1
};

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

void loop() {
  for (int i = 0; i < 8; i++) { // 8 pasos por ciclo
    digitalWrite(IN1, paso[i][0]);
    digitalWrite(IN2, paso[i][1]);
    digitalWrite(IN3, paso[i][2]);
    digitalWrite(IN4, paso[i][3]);
    delay(10);
  }
}
```

Versión B — Hardcodeada (Unipolar_Step_Half_HC)

```
#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11
```

```
void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

void loop() {
  // Paso 1: solo IN1
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW); delay(10);

  // Paso 2: IN1 + IN2
  digitalWrite(IN1, HIGH); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW); delay(10);

  // Paso 3: solo IN2
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW); delay(10);

  // Paso 4: IN2 + IN3
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW); delay(10);

  // Paso 5: solo IN3
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW); delay(10);

  // Paso 6: IN3 + IN4
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, HIGH); delay(10);

  // Paso 7: solo IN4
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH); delay(10);

  // Paso 8: IN4 + IN1
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH); delay(10);
}
```

⚠ ¿Qué observar al comparar los tres modos?

- Carga cada modo uno a la vez y observa la suavidad del giro y la resistencia del eje.
- El Half Step produce un giro más suave que los otros dos — esto se debe a que los pasos son más pequeños.
- Experimenta reduciendo el delay(10) en cada modo — cada uno tendrá un punto donde el motor empieza a perder pasos.
- Para invertir el sentido de giro sin cambiar el hardware, recorre la matriz en orden inverso (i de 3 a 0, o de 7 a 0 en Half Step).

⚠ Versión con matriz vs. versión hardcodeada — ¿qué diferencia hay en el código?

- Ambas versiones producen exactamente el mismo comportamiento en el motor.
- La versión con matriz es más compacta y flexible: cambiar de modo solo requiere modificar la matriz.
- La versión hardcodeada hace explícito cada estado de cada bobina en cada paso — útil para entender el principio de operación sin abstracción.